When Does the GPU Pay Off? Checkpoint 2 Milestone Report

Christian Joseph Bunyi
De La Salle University
Manila, Philippines
christian_bunyi@dlsu.edu.ph

Enzo Rafael Chan
De La Salle University
Manila, Philippines
enzo_rafael_chan@dlsu.edu.ph

Immanuel Umali
De La Salle University
Manila, Philippines
immanuel_umali@dlsu.edu.ph

Abstract

We are measuring when a CUDA implementation of norm-scaled stochastic quantization is faster than a single-thread C implementation on one RTX 5060. At Week 10, the C codec packs and decodes at 4 and 8 bits, and the CUDA scale bits and signed codes match C byte for byte at both widths. Under the course benchmark protocol, with host-to-device and device-to-host copies counted, CUDA is 8.3 to 19.9 times faster than C at 2^{18} and 2^{22} elements, and slower at 2^{10} (0.07 times as fast). Host-side packing of CUDA-generated codes was not demonstrated, because GPU packing replaced it. Timing 2^{14} to complete the four-size comparison, further decoder and edge-case checks, and the final demonstration remain for Week 13.

Keywords

Stochastic Quantization, CUDA, Randomized Rounding, Milestone Report

1 The Project

Compressing a model update before sending it reduces the bytes uploaded, but the compression costs time on the client. We measure that time for norm-scaled stochastic quantization, a QSGD-style scheme whose randomized rounding is unbiased in ideal arithmetic [1]. The CUDA pipeline has three kernels. K1 reduces the tensor to its scale with a fixed pairwise tree, so C and CUDA add in the same order [2]. K2 rounds each element with counter-based Philox random words [3]. K3 packs the codes into 4-bit or 8-bit form behind a 20-byte header. A single-thread C implementation is the comparator and follows the same reduction order, rounding arithmetic, and random words. The project covers compression and reconstruction of synthetic tensors. Federated training, model accuracy, and network transmission are outside its scope.

2 Milestone Commitments and Evidence

The proposal set three checks for Week 10, which Table 1 splits into six commitments. Five are met, and host-side packing is not.

3 Timings

We ran `uv run python -m stoquant bench-matrix --counts 1024 262144 4194304 --bits 4 8` on a clean checkout of course `main` at b2f6b09. Each of four paths ran in 24 independent processes with 10 warm-up and 30 measured repetitions, at least 1 s of in-process warm-up, and balanced path orders. The input is normal FP32 from `numpy.random.default_rng`, seeded with the element count and 2026. The *host-origin* path copies the input to the GPU and the record back from pageable memory, and the *resident* path keeps the data on the GPU. Each time is the median over all repetitions, and the host-origin column also gives the pooled interquartile range (IQR). Speedup is the C median divided by the host-origin median. The range combines the trial extremes: its lower bound is the smallest C trial median divided by the largest host-origin trial median, and its upper bound is the largest C trial median divided by the smallest host-origin trial median. K1 and K2 are CUDA-event stage medians of the resident path; they diagnose a path and are not summed into a path time.

With copies counted (Table 2), CUDA is 8.3 to 8.4 times faster than C at 2^{18} and 18.4 to 19.9 times faster at 2^{22}. At 2^{10}, C wins: its median is 0.014 to 0.015 ms against 0.206 ms for host-origin CUDA, so CUDA runs at 0.07 times the speed of C. The resident path, which copies nothing, takes 0.106 and 0.111 ms at 2^{10}, and so also loses to C there. At 2^{18} and 2^{22} it is 24.8 to 26.2 times and 157 to 159 times faster than C. The snapshot also holds a fourth, resident-graph path that the table omits.

We claim a speedup or slowdown only when the cell passes revision 3 of the protocol's claim rules. A cell needs `direction_supported` and `magnitude_supported` true and `boundary_inversion` false. The magnitude test requires the ratio of the 90th to the 10th percentile across trials to be at most 1.25. All six host-origin cells pass. Across the 18 CUDA cells, 17 pass; resident-graph at 2^{10} and 8 bits is inconclusive and gets no claim. The column `claim_supported_rev2` applies an older and stricter rule, a maximum-to-minimum trial spread of at most 1.25, and it fails for three of the six host-origin cells; we apply revision 3. The snapshot's `manifest.json` has SHA-256 `2b2fe40a. . . 9c96` and its `summary.csv` has SHA-256 `b6ee4d22. . . 6c2d`; the full hashes are in the course repository's evidence file.

Single-sample K1 timings at 2^{10} from before we adopted the protocol did not follow input size: 1.78 ms at 4 bits and 0.795 ms at 8 bits. The snapshot gives 0.057 ms at both widths. We think per-process cold start caused the earlier values, and we have not tested that.

4 Deviation and Progress Beyond the Milestone

The Week 10 plan named host-side packing of CUDA-generated codes, and we did not demonstrate it. GPU packing in K3 was the stretch goal, and the built pipeline packs on the GPU, so the host decodes the GPU's record and never packs. The substitute evidence is that the CUDA and C records are byte-identical at both widths, in 100 parametrized test cases and in all 24 benchmark cases. GPU packing is progress beyond the milestone; `test_cuda_k3_4bit_packing_structure_and_odd_length` covers it.

5 Limits

The measurements come from one NVIDIA GeForce RTX 5060 (compute capability 12.0, driver 610.88, CUDA 13.4, MSVC 19.51) on Windows, and the same GPU drives the display. Transfers use pageable host memory. The comparator is a scalar single-thread C

Table 1: Week 10 commitments and their evidence. The suites ran at 5fd0b78, whose source tree is identical to merged course `main` at b2f6b09.

Commitment	Status	Evidence
Single-thread C pipeline at both widths, with packing, decoding, and exact-code tests	Met	`just test-cpu`: 458 passed, 291 skipped. Prescribed scale and random words fix every expected code (`test_compress_matches_prescribed_scale_and_word_record` and its 4-bit counterpart); native `test_codec` and `test_quantizer` pass.
CUDA scale bits and signed codes match C at both widths	Met	100 parametrized cases show byte-identical CPU and CUDA records at 4 and 8 bits (`test_cuda_record_is_byte_identical_to_cpu`). In `just test-cuda`, the native CUDA API test passed and 307 pytest cases passed. The benchmark correctness gate repeats the check at 2^{10}, 2^{18}, and 2^{22} for both widths: 24 of 24 cases byte-identical.
Reconstruction within the FP64 oracle bounds	Met	`test_cuda_layer_two_decode_satisfies_fp64_bound` and the CPU layer-two tests pass. The same gate checks scale and reconstruction against the FP64 bound in all 24 cases.
Host decoding of CUDA-generated codes	Met	The command-line tool decodes the GPU record on the host, as the demo shows. The FP64 bound tests in the reconstruction row cover the same record.
Scale and rounding timings at 2^{10} and 2^{18} elements	Met	K1 and K2 stage medians, and four path timings, under the course protocol (Section 3). The protocol also covers 2^{22}.
Demo of CUDA-generated codes reconstructed with host-side packing	Not demonstrated	Section 4.

Table 2: Medians in milliseconds from the benchmark snapshot, with the host-origin IQR and speedup bounds computed from the trial extremes.

		C	CUDA resident			CUDA host-origin			
Elements	Bits		total	K1	K2	median	IQR	vs C	range
1,024 (2^{10})	4	0.0153	0.106	0.057	0.010	0.206	0.041	0.07×	0.062 to 0.085
1,024 (2^{10})	8	0.0139	0.111	0.057	0.010	0.206	0.032	0.07×	0.062 to 0.15
262,144 (2^{18})	4	4.07	0.164	0.114	0.011	0.490	0.050	8.3×	7.6 to 9.0
262,144 (2^{18})	8	4.29	0.164	0.115	0.011	0.509	0.053	8.4×	6.9 to 14.7
4,194,304 (2^{22})	4	65.2	0.414	0.289	0.085	3.28	0.038	19.9×	19.4 to 21.3
4,194,304 (2^{22})	8	66.0	0.416	0.289	0.085	3.60	0.021	18.4×	18.2 to 18.7

implementation, so the speedups overstate the gain over an optimized CPU implementation. Warm-up reduces between-process instability but does not remove it, which is why Table 2 lists the range. The demo clip prints one timings line from a single run; it is a demonstration and not evidence. The A100 extension is out of scope for this checkpoint. An independent rerun by a second member on another NVIDIA GPU has not been done.

6 Remaining Work

The winner changes somewhere between 2^{10} and 2^{18}, and this checkpoint does not say where, because 2^{14} is not yet timed. The Week 13 deliverable times 2^{14} under the same protocol to complete the four-size comparison, and adds further decoder and edge-case checks, run instructions, and the final demonstration. Chan will also rerun the checks independently on a second NVIDIA GPU.

7 Where to Find the Work

The code is in the public course repository at revision b2f6b09: https://github.com/qu1r0ra/CSC612M-MCO2-Code. The benchmark snapshot is under `results/2026-10-29-b2f6b09/`, and the evidence file at the repository root describes it. The presentation video will accompany this report when we submit it.

References

[1] Dan Alistarh, Demjan Grubic, Jerry Li, Ryota Tomioka, and Milan Vojnovic. 2017. QSGD: Communication-Efficient SGD via Gradient Quantization and Encoding. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper/2017/hash/6c340f25839e6acdc73414517203f5f0-Abstract.html

[2] Mark Harris. 2007. Optimizing Parallel Reduction in CUDA. NVIDIA Developer Technology presentation. https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf

[3] John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. 2011. Parallel Random Numbers: As Easy as 1, 2, 3. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*. ACM, 1–12. doi:10.1145/2063384.2063405